

Session 09: ADC Calibration with ESP 32Date of the Session: 10 / 1 / 15

Time of the Session: _____ to _____

PREREQUISITE:

- General idea of ESP32 board
- General idea of Analog to Digital Converter (ADC)

PRE-LAB:

1. What is ADC and where it can be used?
ADC converts analog to digital - Used in sensors, audio etc
2. How many ADC pins are there in ESP 32?
18
3. List out the features of ADC in ESP 32.
12-bit resolution, multiple channels, low power

OBJECTIVE:

To ensure precise and consistent conversion of analog signals into accurate digital values.

REQUIRED COMPONENTS:

- ESP32 development board
- Connecting Wires
- Breadboard

THEORY:**ESP32 ADC Calibration:**

The ESP32 Arduino Core ADC driver's API provides functions to correct for differences in measured voltages caused by variation of ADC reference voltages (VREF) between ESP32 chips.

Correcting ADC readings using this API involves characterizing one of the ADCs at a given attenuation to obtain a characteristics curve (ADC-Voltage curve) that takes into account the difference in ADC reference voltage.

The characteristics curve is in the form of $y = \text{coeff_a} * x + \text{coeff_b}$ and is used to convert ADC readings to voltages in mV. Calculation of the characteristics curve is based on calibration values which can be stored in eFuse or provided by the user.

CONNECTION PROCEDURE:

Make sure to connect the potentiometer to that analog channel pin and tweak the pot until you get 2v (or any other value). Just make sure the DMM is reading an exact value (2v or any other value).

Now, read the ADC with the `analogRead()` function without calibration or whatsoever and note down that value. For me, it was 1.85v, while the DMM is reading an exact 2v. Then, incorporated the ADC calibration functions in the code.

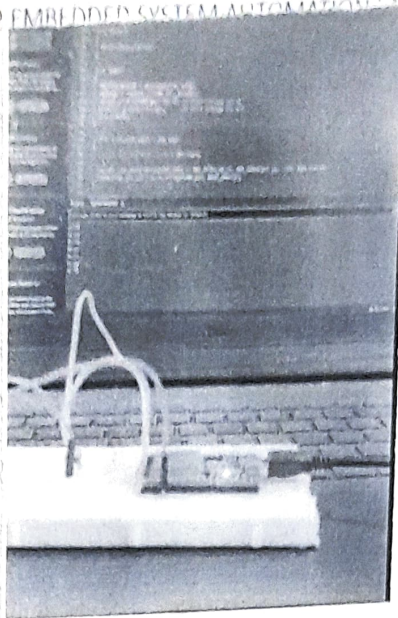
CODE:

```
#include "esp_adc_cal.h"
#define AN_POT1 35
int AN_POT1_Result = 0;
float Voltage = 0.0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  AN_POT1_Result = analogRead(AN_POT1);
  Voltage = readADC_cal(AN_POT1_Result);
  Serial.println(Voltage / 1000.0);
  delay(1000);
}

uint32_t readADC_cal(int ADC_raw) {
  esp_adc_cal_characteristics_t adc_chars;
  esp_adc_cal_characteristics_t adc_chars;
  return 1;
}
```

OUTPUT:**POST LAB**

Callibrated ADC for accurate readings

INTERFERENCE & ANALYSIS

No significant interference observed

RESULT:

Callibration improved accuracy